

Министерство науки и высшего образования Российской Федерации

Московский физико-технический институт
(национальный исследовательский университет)

Физтех-школа прикладной математики и информатики

ОТЧЁТ

по заданию 02 «Ускорение обучения»

курса «Введение в эффективные системы машинного обучения»

Выполнил: Дьячковский Михаил

Москва, 2026

Содержание

1	Постановка задачи и условия экспериментов	2
2	Автоматическая смешанная точность	2
2.1	Результаты	2
3	Эффективная загрузка данных	3
3.1	Результаты	3
4	Профилирование	4
4.1	Методика	4
4.2	Исходный профиль	4
4.3	Найденные неэффективности и их устранение	5
4.4	Вклад исправлений	8
4.5	Профиль после исправлений	9
5	Дополнительные эксперименты	10
5.1	torch.compile	11
5.2	FusedLayerNorm из Apex	11
6	Итоги	11
A	Состав архива с кодом	12

1 Постановка задачи и условия экспериментов

Цель работы состоит в ускорении обучения GPT-2-подобной языковой модели на корпусе WikiText-103 без потери качества. Работа включает четыре этапа: реализацию функции потерь, метрики и позиционного кодирования; рукописную реализацию автоматической смешанной точности; реализацию трёх способов формирования батчей; профилирование выбранной конфигурации и устранение найденных неэффективностей. Дополнительно исследованы компиляция модели средствами `torch.compile` и замена слоёв нормализации на `FusedLayerNorm` из библиотеки Apex.

Модель `GPT2LikeModel` задана условием и состоит из таблицы вложений размера 30522×512 , позиционного кодирования, линейного слоя $512 \rightarrow 1020$, одного слоя `TransformerDecoderLayer` с размерностью 1020 и четырьмя головами внимания и выходного линейного слоя $1020 \rightarrow 30522$. Общее число параметров модели равно 60,9 млн, причём 46,8 млн из них приходится на таблицу вложений и выходной слой. Это обстоятельство существенно для дальнейшего: операции, выполняемые над всеми параметрами на каждом шаге, стоят одинаково при любом размере батча.

Все измерения выполнены в среде Kaggle на одном ускорителе Tesla T4 (Python 3.13.15, PyTorch 2.11.0 с CUDA 12.8, transformers 5.16.1, numpy 2.1.3). Обучение во всех экспериментах длится одну эпоху на валидационной части корпуса, оптимизатор Adam с шагом 10^{-4} , размер батча 32, значение генератора случайных чисел зафиксировано равным 42. Токенизатор `bert-base-uncased`, максимальная длина последовательности 512 токенов.

Время на GPU измерялось парой событий `torch.cuda.Event`: первое событие записывается перед измеряемым участком, второе после него, затем вызывается `torch.cuda.synchronize()` и вычисляется `start.elapsed_time(end)`. Время подготовки данных, которая выполняется только на центральном процессоре, измерялось функцией `time.perf_counter`. Пиковая память определялась функцией `torch.cuda.max_memory_allocated` после сброса счётчика в начале запуска. Качество в таблицах приводится как среднее значение функции потерь за последние 20 итераций эпохи. Каждое измерение выполнено один раз, поэтому различия порядка одного–двух процентов нельзя считать значимыми.

2 Автоматическая смешанная точность

2.1 Результаты

Сравнивались четыре режима при исходном способе загрузки данных: одинарная точность (`fp32`), модель, целиком переведённая в половинную точность без масштабирования (`fp16`), и смешанная точность со статическим и динамическим масштабировщиком. Измерялось время вычисления градиентов, то есть прямого и обратного прохода, на одной итерации. Результаты приведены в таблице 1.

Таблица 1 – Сравнение режимов точности, 77 итераций

Режим	Итераций с <code>nan</code>	Loss	Ассурасу, %	Время градиентов, с	Ускорение
<code>fp32</code>	0	6,7949	12,13	$1,510 \pm 0,057$	1,00
<code>fp16</code>	76	<code>nan</code>	0,00	$0,349 \pm 0,008$	4,33
<code>static</code>	0	6,7937	12,15	$0,443 \pm 0,007$	3,41
<code>dynamic</code>	0	6,7937	12,15	$0,443 \pm 0,008$	3,41

Смешанная точность ускоряет вычисление градиентов в 3,41 раза при том же качестве: значение функции потерь отличается от режима `fp32` на 0,001. Прямой перевод модели в половинную точность даёт конечное значение функции потерь только на одной итерации из 77 и потому непригоден. Коэффициент динамического масштабирования за эпоху не изменился: переполнений не было, а эпоха короче порога терпения ($77 < 80$). Кривые обучения приведены на рисунке ??.

Чтобы показать преимущество динамического масштабирования, оба масштабирования запускались с заведомо завышенным начальным коэффициентом $S = 2^{24}$. Со статическим масштабировщиком значение функции потерь в конце эпохи равно 10,48, что близко к значению $\ln 30522 = 10,33$ для равномерного предсказания: коэффициент остался равным 2^{24} , и модель практически не обучалась. Динамический масштабировщик уменьшил коэффициент с 2^{24} до $2^{19} = 524288$, то есть выполнил пять делений пополам, после чего обучение пошло обычным образом и значение функции потерь составило 6,82.

3 Эффективная загрузка данных

3.1 Результаты

Сравнение выполнено в режиме смешанной точности с динамическим масштабировщиком (таблица 2). Со статическим масштабировщиком получены те же времена с точностью до 0,3 %.

Таблица 2 – Сравнение способов формирования батчей, режим `dynamic`

Способ	Итераций	Подготовка, с	Эпоха, с	Пик памяти, МБ	Loss
<code>base</code>	77	0,22	37,82	6380	6,7937
<code>standard</code>	77	0,53	23,62	5942	6,7979
<code>balanced</code>	88	0,55	11,01	6057	6,4774
<code>sequenced</code>	577	0,56	27,94	2200	6,2801

Дополнение до максимума по батчу сокращает эпоху в 1,60 раза относительно исходного способа, а группировка по длине сокращает её в 3,44 раза. Упаковка последо-

вательностей уменьшает пик памяти в 2,9 раза, поскольку на каждом шаге обрабатывается не более 512 токенов, но эпоха при этом состоит из 577 шагов и занимает 27,94 с. Подготовка данных в трёх реализованных способах дольше на 0,3 с, так как весь корпус токенизируется заранее; это время возвращается уже на первых итерациях. Значения функции потерь между способами напрямую не сравнимы: за эпоху выполняется разное число шагов оптимизатора (77, 88 и 577).

Абсолютные значения пика памяти в этой и следующих таблицах включают память, которая оставалась занятой в той же сессии после предыдущих запусков. Одна и та же конфигурация дала 2200 МБ в таблице 2 и 3120 МБ в таблице 5, поэтому сравнивать следует только строки внутри одной таблицы.

Для дальнейшей работы зафиксирована конфигурация **sequenced** с динамическим масштабировщиком. Она требует наименьшей памяти, не тратит вычисления на токены заполнения и за эпоху достигает наименьшего значения функции потерь. Её недостаток, большая длительность эпохи по сравнению со способом **balanced**, как показано ниже, вызван постоянными издержками каждого шага и в значительной мере устраняется.

4 Профилирование

4.1 Методика

Профиль снимался средствами `torch.profiler` по образцу второго семинара курса. Использовано расписание `schedule(wait=2, warmup=2, active=5, repeat=1)`: первые два шага пропускаются, следующие два служат для прогрева, пять шагов записываются. Включены запись форм тензоров, профилирование памяти и запись стека. Функция `profiler.step()` вызывается в конце каждой итерации обучения, а обучение останавливается после девяти шагов. Обработчик `tensorboard_trace_handler` сохраняет трассу в файл `*.pt.trace.json`, который открывается в Perfetto.

Для количественного разбора та же трасса читается функцией `summarize_trace`. Она отбирает завершённые события, определяет длительность шага по аннотациям `ProfilerStep` и суммирует число вызовов и длительность по категориям: операции PyTorch на процессоре (`cpu_op`), ядра GPU (`kernel`), вызовы среды исполнения CUDA (`cuda_runtime`) и копирования памяти. Для сравнения профилей до и после исправлений время ядер GPU относится к той операции PyTorch, которая их запустила.

4.2 Исходный профиль

До исправлений шаг обучения в выбранной конфигурации длится 51,7 мс, из которых ядра GPU заняты 43,4 мс (84 %). Основные статьи профиля приведены в таблице 3.

Таблица 3 – Исходный профиль: основные статьи на один шаг обучения

Статья	Вызовов на шаг	Время, мс
Шаг целиком	—	51,7
Ядра GPU, всего	455	43,4
Шаг оптимизатора <code>Adam.step</code> , GPU	1	18,1
<code>cudaStreamSynchronize</code> , ожидание GPU	9	23,9
<code>aten::item</code> , процессор	51	20,7
<code>aten::nonzero</code> , процессор	3	3,8
<code>cudaDeviceSynchronize</code>	1,2	1,6
Перенос данных на GPU	3	0,02

Из профиля следуют два наблюдения. Во-первых, сама модель занимает небольшую часть времени GPU: матричные умножения (`aten::mm`, `aten::addmm`, `aten::bmm`) дают 5,9 мс из 43,7 мс. Остальное время уходит на операции, которые проходят по всем 60 млн параметров: обновление оптимизатора, обнуление и накопление градиентов, проверку и масштабирование градиентов. Стоимость этих операций не зависит от размера батча, а конфигурация `sequenced` выполняет их 577 раз за эпоху; только шаг оптимизатора даёт $18,1 \text{ мс} \times 577 \approx 10,4 \text{ с}$ из 27,9 с. Этим объясняется, почему эпоха в этой конфигурации длиннее, чем в конфигурации `balanced` с 88 шагами. Во-вторых, процессор девять раз за шаг останавливается в ожидании GPU. Само же копирование данных на GPU занимает 0,02 мс.

4.3 Найденные неэффективности и их устранение

Каждое исправление включается отдельным флагом. При выключенных флагах исправленный код повторяет исходный, что подтверждается первой строкой таблицы 5: 27,79 с против 27,94 с в таблице 2. Исправления включались по одному в порядке убывания ожидаемого эффекта; в числах ниже «до» и «после» относятся к профилям без исправлений и со всеми исправлениями, а изменение времени эпохи относится к включению одного данного исправления.

1. Шаг оптимизатора в несколько проходов. Оптимизатор Adam по умолчанию выполняет обновление семью отдельными операциями над списком всех параметров (таблица 4). В сумме они занимают 18,1 мс, или 41 % времени GPU.

Таблица 4 – Операции шага оптимизатора в исходном профиле, время GPU на шаг

Операция	Время, мс	Операция	Время, мс
<code>aten::_foreach_addcddiv_</code>	3,85	<code>aten::_foreach_div_</code>	2,08
<code>aten::_foreach_lerp_</code>	2,96	<code>aten::_foreach_add_</code>	2,07
<code>aten::_foreach_addcmul_</code>	2,96	<code>aten::_foreach_mul_</code>	2,07
<code>aten::_foreach_sqrt</code>	2,09	Всего	18,08

Каждая операция заново читает и записывает все параметры, тогда как всё обновление можно выполнить за один проход. Исправление состоит в передаче оптимизатору аргумента `fused=True`:

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4,
                              fused=True if args.fused_adam else None)
```

После исправления семь операций заменяет одна операция `_fused_adam_` длительностью 7,3 мс. Эпоха сократилась с 27,79 до 21,83 с, то есть в 1,27 раза.

2. Обнуление градиентов записью нулей. Градиенты обнуляются вызовом `zero_grad` с аргументом `set_to_none=False`, который записывает нули во все тензоры градиентов: в профиле это 38 вызовов `aten::fill_` на шаг и 1,07 мс внутри `Optimizer.zero_grad`. Кроме того, поскольку тензоры градиентов продолжают существовать, обратный проход не присваивает новый градиент, а прибавляет его к нулям: 55 вызовов `aten::add_` и 3,15 мс на шаг. Обе операции лишние. При `set_to_none=True` градиенты удаляются, и обратный проход записывает их заново. После исправления число вызовов `aten::add_` упало до 8, а их время до 0,17 мс; время `aten::fill_` упало с 1,76 до 0,64 мс. Эпоха сократилась с 21,83 до 19,81 с (1,10 раза), пик памяти уменьшился на 104 МБ.

3. Проверка конечности градиентов в несколько проходов. Масштабировщик перед шагом проверял каждый из 23 тензоров градиентов функцией `torch.isfinite`:

```
all_finite = torch.stack([torch.isfinite(grad).all() for grad in grads]).all().item()
```

В профиле видно, что эта функция составная: на каждый градиент выполняются операции `aten::abs`, `aten::ne`, `aten::eq`, `aten::mul` и `aten::all`, каждая со своим проходом по тензору и временным тензором. Операции `abs`, `ne`, `eq` и `all` вместе занимали 4,8 мс на шаг. Для проверки достаточно одного прохода: сумма элементов тензора неконечна, если хотя бы один элемент неконечен, поскольку значение `nan` сохраняется при сложении, а бесконечности разных знаков дают `nan`.

```
def all_finite_one_pass(grads):
    return torch.stack([grad.sum(dtype=torch.float32) for grad in grads]).isfinite().all().item()
```

Обратное неверно только при переполнении суммы конечных слагаемых; в этом случае шаг будет пропущен, что безопасно. После исправления операции `abs`, `ne`, `eq` и `all` практически исчезли из профиля (0,03 мс), а время `aten::sum` выросло с 0,26 до 1,27 мс. Эпоха сократилась с 19,81 до 17,15 с (1,15 раза), пик памяти уменьшился на 125 МБ, поскольку временные тензоры больше не создаются.

4. Функция потерь с индексацией булевой маской. Выражения `logits[mask]` и `targets[mask]` требуют списка индексов ненулевых элементов маски. Его размер заранее неизвестен, поэтому операция `aten::nonzero` ждёт завершения всех предыдущих вычислений на GPU: три вызова на шаг, 3,8 мс ожидания на процессоре. Далее `aten::index` копирует выбранные строки логитов (0,40 мс), а в обратном проходе `aten::_index_put_impl_` разносит градиент обратно в нулевой тензор размера логитов (0,36 мс). Исправление использует логиты целиком, а недопустимые переходы исключает значением `ignore_index` в целевых токенах:

```
targets = targets.masked_fill(mask.logical_not(), self.ignore_index)
return super().forward(logits.flatten(0, 1), targets.flatten())
```

Значение функции потерь при этом то же, так как усреднение выполняется по тем же переходам. После исправления операции `aten::nonzero`, `aten::index` и `aten::_index_put_impl_` в профиле отсутствуют. Эпоха сократилась с 17,15 до 16,40 с (1,05 раза).

5. Синхронизация с GPU на каждом шаге. На каждом шаге вызывались `loss.item()` и `accuracy.item()` для строки прогресса, метрика вычислялась на каждом шаге, а для замера времени вызывалась `torch.cuda.synchronize()`. Вызов `.item()` копирует значение с GPU и потому ждёт завершения всех поставленных в очередь вычислений; пока процессор ждёт, он не готовит следующий батч. В исходном профиле на шаг приходится 9 вызовов `cudaStreamSynchronize` с суммарным ожиданием 23,9 мс. Их состав восстанавливается по числу копирований: три копирования на GPU при переносе батча и шесть копирований с GPU, из которых три относятся к `aten::nonzero`, два к вызовам `.item()` для строки прогресса и одно к проверке градиентов в масштабировщике. После исправления значение функции потерь на каждом шаге сохраняется как тензор на GPU без копирования и считывается один раз в конце эпохи; метрика и строка прогресса обновляются раз в 10 шагов; события `torch.cuda.Event` записываются на каждом шаге, но считываются после единственной синхронизации в конце эпохи. Вместе с предыдущим исправлением, убравшим три вызова `aten::nonzero`, это сократило число вызовов `cudaStreamSynchronize` с 9 до 4,2 на шаг, время ожидания с 23,9 до 8,6 мс, число копирований с GPU с 6 до 1,2, а время `aten::item` с 20,7 до 2,4 мс. Эпоха сократилась с 16,40 до 15,58 с (1,05 раза).

6. Неиспользуемый линейный слой. В прямом проходе модели вычисляется `y = self.hidden_projector2(x)`, результат которого нигде не используется. Вычисление исключено; в профиле число вызовов `aten::linear` и `aten::addmm` уменьшилось с 10 до 9 на шаг. Измеримого ускорения это не дало (15,58 и 15,57 с): слой размера 1020×1020 применяется не более чем к 512 токенам и не участвует в обратном проходе.

7. Построение причинной маски на каждом шаге. Если загрузчик не передаёт маску внимания, модель на каждом прямом проходе заново строит треугольную маску и преобразует её в аддитивную. В исправленной модели аддитивная маска строится один раз и хранится как буфер. В выбранной конфигурации маска зависит от состава упаковки и приходит из набора данных, поэтому это исправление на её скорость не влияет и в таблицу 5 не включено.

8. Размерность скрытого слоя, не кратная восьми. Проверена замена размерности 1020 на 1024. Эпоха сократилась с 15,57 до 15,20 с (1,02 раза), что сопоставимо с разбросом одиночных измерений. Поскольку при этом меняется сама модель (значение функции потерь 6,50 против 6,28 при другой инициализации), замена в итоговую конфигурацию не включена.

4.4 Вклад исправлений

Вклад каждого исправления во время эпохи приведён в таблице 5 и на рисунке 1. Каждая строка получена из предыдущей включением одного исправления, поэтому величина вклада зависит от порядка включения.

Таблица 5 – Последовательное включение исправлений, 577 шагов

Шаг	Эпоха, с	мс/шаг	К пред.	К исх.	Память, МБ	Loss
Исходный код	27,79	48,2	1,00	1,00	3120	6,2801
+ fused Adam	21,83	37,8	1,27	1,27	3122	6,2792
+ zero_grad(set_to_none=True)	19,81	34,3	1,10	1,40	3018	6,2792
+ проверка градиентов за один проход	17,15	29,7	1,15	1,62	2893	6,2792
+ loss без булевой индексации	16,40	28,4	1,05	1,69	2893	6,2799
+ .item() и ассигасу раз в 10 шагов	15,58	27,0	1,05	1,78	2893	6,2799
+ без hidden_projector2	15,57	27,0	1,00	1,79	2894	6,2799
+ размерность 1024 (другая модель, 581 шаг)	15,20	26,2	1,02	1,83	2874	6,5045

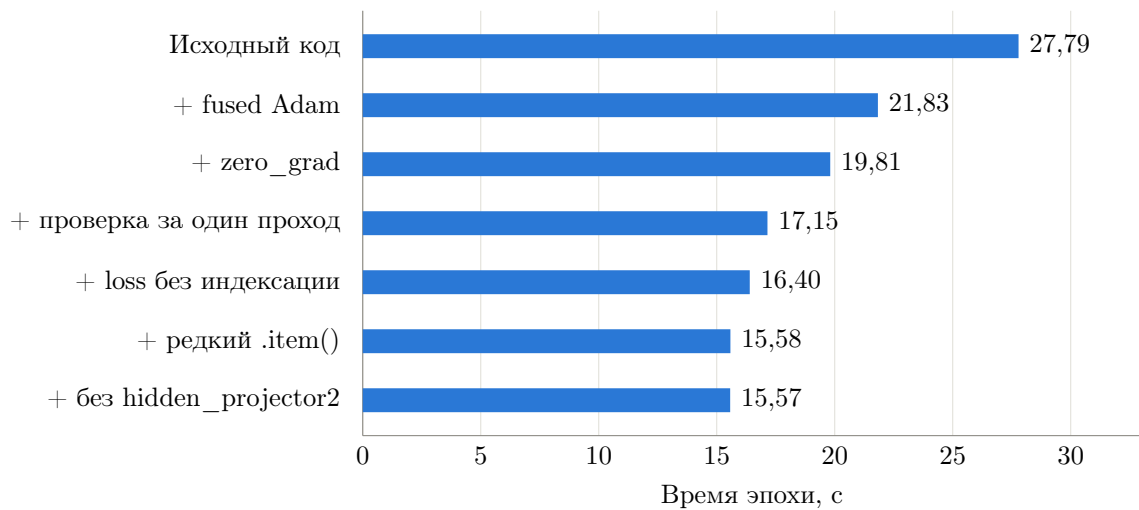


Рисунок 1 – Время эпохи при последовательном включении исправлений

При неизменной модели эпоха сократилась с 27,79 до 15,57 с, то есть в 1,79 раза, а пик памяти уменьшился на 226 МБ. Значение функции потерь изменилось в четвёртом знаке (6,2801 и 6,2799), что объясняется иным порядком арифметических операций в объединённом шаге оптимизатора и в функции потерь.

4.5 Профиль после исправлений

Профиль со всеми исправлениями снят по тому же расписанию с десятью записанными шагами, чтобы в запись попал один шаг с вычислением метрики. Шаг длится 30,2 мс вместо 51,7 мс, ядра GPU заняты 25,0 мс (83 %). Суммарное время GPU по операциям уменьшилось с 43,7 до 24,5 мс на шаг. Распределение этого времени до и после исправлений показано на рисунке 2.

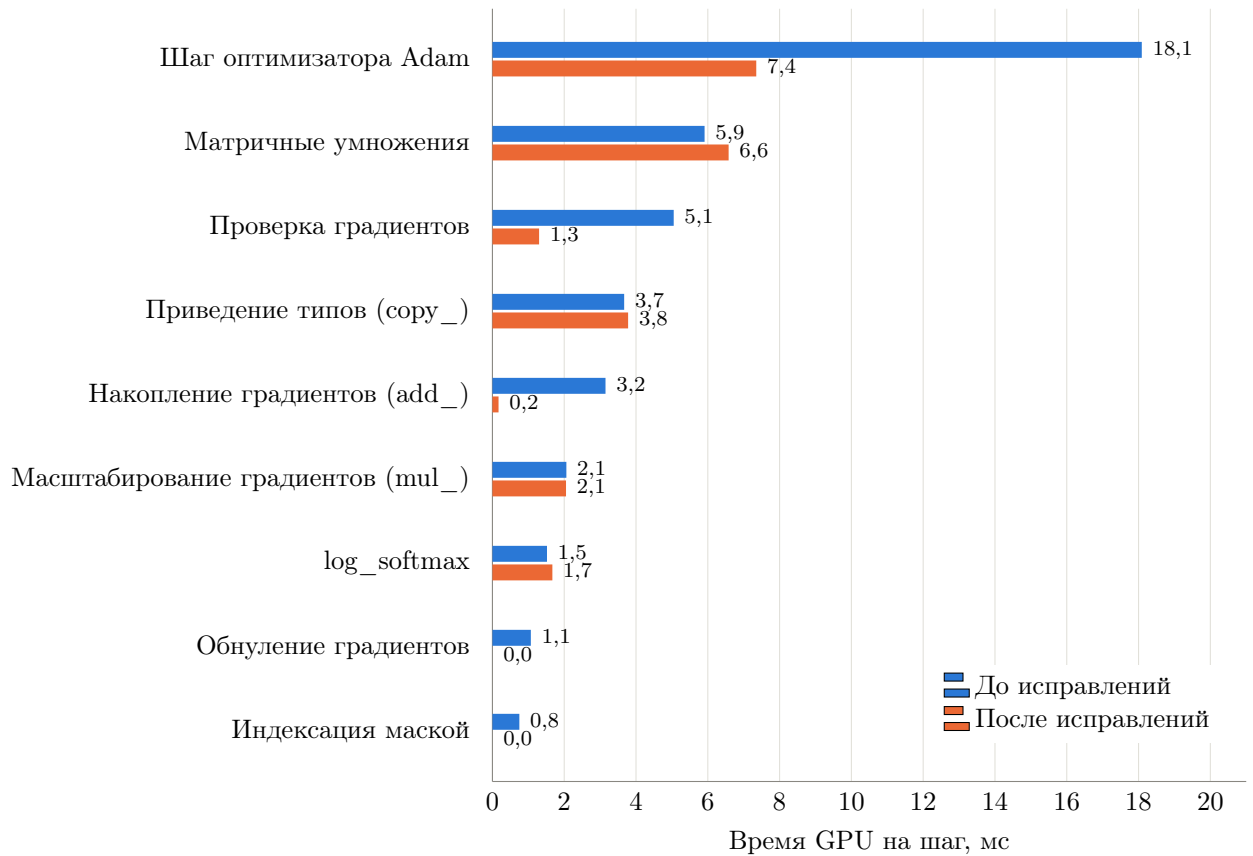


Рисунок 2 – Время GPU на один шаг по группам операций. Матричные умножения: `aten::mm`, `aten::addmm`, `aten::bmm`; проверка градиентов: `aten::abs`, `aten::ne`, `aten::eq`, `aten::all`, `aten::sum`; обнуление градиентов: аннотация `Optimizer.zero_grad`; индексация маской: `aten::index`, `aten::_index_put_impl`

Оставшееся время GPU распределено следующим образом: объединённый шаг оптимизатора 7,3 мс, матричные умножения модели 6,6 мс, приведение весов к половинной точности в **Autocast** 3,8 мс, деление градиентов на коэффициент масштаба 2,1 мс, логарифм софтмакса в прямом и обратном проходе 1,7 мс, проверка градиентов 1,3 мс. Время матричных умножений в двух профилях различается (5,9 и 6,6 мс), хотя исправления их не затрагивали; это даёт оценку разброса между отдельными записями профиля.

В профиле остаётся 4,2 ожидания GPU на шаг. Одно из них необходимо: масштабировщик должен узнать результат проверки градиентов, прежде чем решить, выполнять ли шаг. Три других связаны с блокирующим переносом тензоров из обычной памяти процессора на GPU (три копирования `Memcpy HtoD (Pageable -> Device)` на шаг; вызовы `aten::to` длительностью до 7,7 мс). Закрепление памяти и неблокирующий перенос в работе не исследовались.

5 Дополнительные эксперименты

Оба эксперимента выполнены в конфигурации со всеми исправлениями. Обучение длилось три эпохи, чтобы отделить время компиляции от установившегося режима.

Результаты приведены в таблице 6.

Таблица 6 – Компиляция модели и замена слоёв нормализации

Вариант	Эпоха 1, с	Эпоха 2, с	Эпоха 3, с	Пик памяти, МБ	Loss
Все исправления	15,88	15,65	15,52	3817	5,4679
+ <code>torch.compile</code>	44,56	14,89	14,97	3817	5,4602
+ <code>FusedLayerNorm</code>	15,90	15,91	15,93	3817	5,4731

5.1 `torch.compile`

Модель компилировалась вызовом `torch.compile(model, dynamic=True)`. Динамические формы необходимы, поскольку длина упаковки меняется от шага к шагу. Первая эпоха заняла 44,56 с, то есть компиляция добавила около 28,7 с. В установившемся режиме эпоха длится в среднем 14,93 с против 15,59 с без компиляции: ускорение составляет 1,04 раза. Пик памяти не изменился. При таком выигрыше, 0,66 с на эпоху, компиляция окупается примерно через 44 эпохи. Небольшой эффект согласуется с профилем: компилируется только прямой и обратный проход модели, а на них приходится меньшая часть шага.

5.2 `FusedLayerNorm` из `Apex`

Из исходных текстов `Apex` собрано только расширение слоя нормализации (файлы `layer_norm_cuda.cpp` и `layer_norm_cuda_kernel.cu`) средствами `torch.utils.cpp_extension.load`. Три слоя `LayerNorm` декодера заменены на `FusedLayerNorm` с переносом весов. Перед вызовом вход приводится к типу весов, то есть к одинарной точности: так сохраняется то же правило, что и в `Autocast`, где нормализация выполняется в одинарной точности.

Ускорения замена не дала: эпоха длится 15,90–15,93 с против 15,52–15,88 с, пик памяти тот же. В профиле обратный проход нормализации занимает 0,15 мс на шаг (`apex::fused_layer_norm_affine_bwd`, три вызова) против 0,12 мс у штатной реализации (`aten::native_layer_norm_backward`). Нормализация применяется три раза за шаг к тензору из не более чем 512 строк и занимает менее одного процента времени шага, поэтому её реализация на длительность обучения в данной конфигурации не влияет.

6 Итоги

Вклад этапов работы сведён в таблицу 7.

Таблица 7 – Ускорение по этапам

Этап	Измеряемая величина	Ускорение
Смешанная точность	время градиентов на итерацию: 1,510 $\rightarrow$ 0,443 с	3,41
Упаковка последовательностей	время эпохи: 37,82 $\rightarrow$ 27,94 с; память: 6380 $\rightarrow$ 2200 МБ	1,35
Устранение неэффективностей	время эпохи: 27,79 $\rightarrow$ 15,57 с	1,79
<code>torch.compile</code>	время эпохи в установившемся режиме: 15,59 $\rightarrow$ 14,93 с	1,04

В режиме смешанной точности эпоха сократилась с 37,82 с при исходной загрузке данных до 15,57 с в итоговой конфигурации, то есть в 2,43 раза, при уменьшении пика памяти почти втрое. Полное время эпохи в одинарной точности отдельно не измерялось, однако одно лишь вычисление градиентов занимает в ней $77 \times 1,510 \approx 116,3$ с. Отсюда следует оценка снизу: относительно исходного кода в одинарной точности обучение ускорено не менее чем в 7,4 раза.

Наибольший вклад дали смешанная точность и объединённый шаг оптимизатора. Основной вывод профилирования состоит в том, что для данной модели время шага определяется не прямым и обратным проходом, а операциями над всеми параметрами: шагом оптимизатора, обнулением, проверкой и масштабированием градиентов. Поэтому наиболее действенными оказались исправления, которые сокращают число проходов по параметрам, а замена слоёв нормализации и удаление неиспользуемого слоя на время обучения не повлияли.

А Состав архива с кодом

Архив повторяет структуру каталога задания. Файл `utils.py` содержит функцию потерь и метрику, `positional_encoding.py` содержит позиционное кодирование, `amp.py` содержит `Autocast` и оба масштабировщика, `data_utils.py` содержит наборы данных, функции сборки батча и сэмплер, `train.py` содержит модель и исходный цикл обучения. Файл `experiments.py` содержит запуски экспериментов, профилирование, исправленный цикл обучения с флагами и дополнительные эксперименты. Файл `notebook9ce6ab5d72.ipynb` есть исполненный блокнот со всеми выводами, на которых основан отчёт.